

Build the *Future* of Agent Trade

A hands-on project to deeply understand how AI agents buy, sell, and transact — by building a working multi-agent commerce system from scratch using real protocols.

A2A · Agent-to-Agent

API · Commerce Payments

UCP · Universal Commerce Protocol

MCP · Model Context Protocol

PROTOCOL 01

A2A

Agent-to-Agent Protocol (Google DeepMind)

A2A defines how two AI agents discover each other, negotiate tasks, and exchange structured messages — without a human in the loop. It uses Agent Cards (JSON descriptors) and standardized push/pull communication.

01

→ In your project: Buyer agent negotiates with seller agent using A2A task handoffs

PROTOCOL 02

API

Agentic Payment Interface (emerging standard)

Agentic Payment Interfaces allow AI agents to autonomously initiate and authorize payments, request quotes, and settle transactions — with human-set spending limits and policy guardrails baked in.

→ In your project: Buyer agent pays for goods using API calls bound by a spending budget

PROTOCOL 03

UCP

Universal Commerce Protocol

UCP is a semantic layer that standardizes how agents describe products, services, availability, and trade terms. It enables machine-readable catalogs that any compliant agent can browse and query.

→ In your project: Seller agent exposes inventory as a UCP-compliant product catalog

PROTOCOL 04

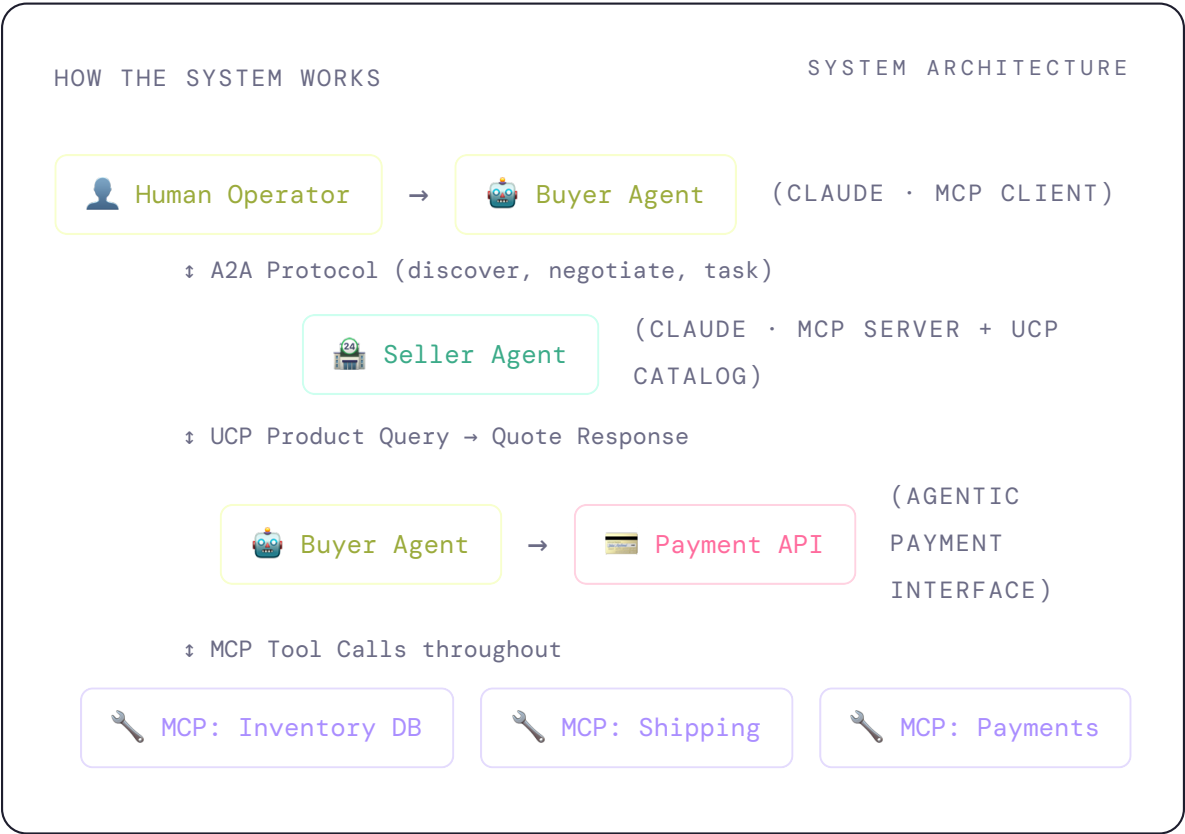
MCP

MCP gives AI models a standardized way to call external tools, APIs, and data sources at inference time. It acts as the nervous system that connects agents to real-world capabilities like databases, shipping APIs, or payment processors.

→ In your project: Both agents use MCP servers to access inventory, payments, and logistics

§01

The Project: SupplyMind Marketplace



You'll build **SupplyMind** — a simulated B2B marketplace where a **Buyer Agent** autonomously sources office supplies from a **Seller Agent**. The buyer discovers the seller via A2A, queries products via UCP, pays via an Agentic Payment Interface, and both agents use MCP servers to access real tools. Each phase adds one protocol layer.

§02 5-Phase Learning Path

FOUNDATION · WEEK 1

1

MCP — Give Your Agents Superpowers

~2-3 days



MCP

Start with MCP because it's the plumbing everything else runs on. You'll build two simple MCP servers and connect a Claude agent to them. By the end, you'll have an agent that can query a database and call a mock API — the foundation for both buyer and seller agents.

1.1 Read the MCP Spec

Study Anthropic's MCP documentation. Understand: tools, resources, prompts, and the client-server model. Focus on how tool schemas are defined.

1.2 Build: Inventory MCP Server

Create a Python MCP server with a SQLite backend. Expose tools:

```
get_product(id) , list_products(category) ,  
check_stock(id) .
```

```
tools: get_product, list_products, check_stock  
backend: sqlite3 with 20 seed products  
transport: stdio or SSE
```

1.3 Build: Shipping MCP Server

A second MCP server exposing: `get_shipping_quote(weight, destination)` and `create_shipment(order_id)` . Stub the responses with realistic fake data.

1.4 Connect Claude as MCP Client

Use the Anthropic SDK with `tool_use` to wire Claude to both MCP servers. Ask it: "Find me 10 units of blue pens and quote shipping to NYC." Watch it call tools end-to-end.

1.5 Milestone

Claude autonomously answers "what's in stock?" and "how much to ship?" by calling your MCP servers. You can trace every tool call in logs.

CATALOG LAYER · WEEK 1-2

2

UCP — Machine-Readable Product Catalog

~2-3 days ▼

UCP MCP

DISCOVERY & NEGOTIATION · WEEK 2

3

A2A — Agents Find & Talk to Each Other

~3-4 days ▼

A2A UCP MCP

COMMERCE LAYER · WEEK 3

4

Agentic Payments — Agents That Pay

~3-4 days ▼

API A2A MCP

INTEGRATION & REFLECTION · WEEK 3-4

5

Full Stack Demo + Protocol Reflection

~2-3 days ▼

A2A API UCP MCP

§03

Use This Prompt to Get Started

MASTER PROMPT · PASTE INTO CLAUDE TO BEGIN

Copy Prompt

I'm building a learning project called SupplyMind — a multi-agent B2B commerce system that teaches me A2A, Agentic Payment Interfaces, UCP, and MCP by building them hands-on.

The project has 5 phases:

Phase 1 (MCP): I'll build two Python MCP servers — one for inventory (SQLite-backed, exposing `get_product`, `list_products`, `check_stock`) and one for shipping (`get_shipping_quote`, `create_shipment`). Then wire Claude as an MCP client to query both.

Phase 2 (UCP): Wrap my inventory server to return UCP-compliant product JSON with pricing tiers, availability, and trade terms. Add a `request_quote` tool that returns a time-bounded UCP Quote object.

Phase 3 (A2A): Create Agent Cards (`agent.json`) for buyer and seller agents. Build a tiny FastAPI registry. Implement A2A task lifecycle on the seller (`POST /tasks/send`, `GET /tasks/{id}`). Wire Claude as the buyer to discover, browse UCP catalog, and submit A2A purchase tasks.

Phase 4 (Agentic Payments): Build a Payment MCP server with `authorize_payment`, `get_balance`, and `get_transaction_history` tools. Enforce spending guardrails (monthly cap, per-transaction limit, merchant allowlist). Complete the loop: buyer pays → notifies seller → seller ships.

Phase 5 (Full Demo): Add a second competing seller, build a simple dashboard, run 5 test scenarios, write a protocol comparison doc.

I want to start with Phase 1 right now. Help me build the Inventory MCP Server in Python. Walk me through it step by step: the SQLite schema, the MCP tool definitions with proper JSON schemas, how to handle the stdio transport, and seed data for 10 office supply products. Show me the full working code.



3 MCP Servers

Inventory, Shipping, and Payment — each a real Python server with typed tool schemas and proper transports.



2 A2A Agents

A buyer agent (Claude) and a seller agent (FastAPI + Claude) that discover and transact autonomously via Agent Cards.



UCP Catalog

A machine-readable product catalog with bulk pricing, availability signals, and time-bounded quote objects.